

个人信息

姓名：胡皓然

学号：2024010671

院系：计算机系

一、实验环境与编码处理（必做）

1.1 实验环境

本实验在以下环境下完成：

- 操作系统：WSL2 (Windows Subsystem for Linux)
- Python 版本：Python 3.13.11
- 开发语言：Python
- 依赖库：仅使用 Python 标准库，无额外依赖

1.2 编码处理方法

本实验涉及多种编码格式的数据文件：

- 拼音汉字表、汉字表以及训练语料均为 GBK 编码
- 程序输出结果为 UTF-8 编码

在程序实现中，采用如下策略处理编码问题：

在读取拼音表、汉字表以及语料时，使用：

```
1 | open(file_path, 'r', encoding='gbk')
```

程序在内存中的字符串统一使用 Python 3 的 Unicode 表示，因此后续统计和解码过程不再依赖原始文件编码。

输出方面，程序将预测得到的汉字句子写到标准输出，并通过命令行重定向保存到

`data/output.txt`。在实验环境下，Python 3 默认以 Unicode 处理字符串，因此只要终端或文件保存环境采用 UTF-8，输出文件即可视为 UTF-8 编码文本。

二、语料库与数据预处理方法（必做）

1. 使用的语料库

本实验实际使用的语料包括：

- 课程提供的必做语料：`sina_news_gbk`
- 额外加入的扩展语料：
 - `SMP2020` :在课程实验中的语料库中提供

2. 语料预处理方法

首先, 程序读取课程提供的“一二级汉字表”, 构造合法汉字集合 `valid_char_table`。该集合中的字符被视为模型允许统计的合法汉字。随后, 程序读取“拼音汉字表”, 建立从拼音到候选汉字列表的映射关系, 用于后续解码时为每个拼音音节生成候选状态。

在遍历语料文件时, 程序按字符扫描文本内容, 仅对出现在合法汉字集合中的字符进行统计。对于不在合法汉字表中的字符, 例如空格、标点、英文字母、数字以及其他符号, 程序一律视为非法字符并跳过, 不参与一元和二元词频统计。同时, 为了避免跨越非法字符错误连接两个本不相邻的汉字, 在遇到非法字符时, 程序会将前一个有效字符状态清空, 从而保证二元统计只发生在连续合法汉字之间。

3. 统计结果的组织方式

经过预处理后, 程序主要得到以下两类统计结果:

- 单字频数表 `uniword_table`: 记录每个合法汉字在训练语料中出现的次数;
- 二元频数表 `bigram_table`: 记录每个相邻汉字对 (w_{i-1}, w_i) 在训练语料中连续出现的次数。(基于二元的情况)

三、基于字的二元模型的拼音输入法（必做）

1. 基本思路、公式推导和算法原理

(1) 基本思路

拼音输入法的目标是: 给定一串拼音序列, 输出最有可能对应的汉字序列。设输入拼音序列为

$$P = p_1, p_2, \dots, p_n$$

希望找到最优汉字序列

$$W = w_1, w_2, \dots, w_n$$

使得该序列在给定拼音条件下的概率最大, 即

$$\hat{W} = \operatorname{argmax}_W P(W | P)$$

根据贝叶斯公式, 有

$$\hat{W} = \operatorname{argmax}_W \frac{P(P|W)P(W)}{P(P)}$$

对于固定输入拼音序列 P , 分母 $P(P)$ 为常数, 因此可以忽略。另一方面, 在拼音汉字转换任务中, 候选汉字序列通常都是由拼音表约束得到的可发音序列, 因此 $P(P | W)$ 在比较不同候选时可以近似看作常量。于是问题可转化为寻找语言模型概率最大的汉字序列:

$$\hat{W} \approx \operatorname{argmax}_W P(W)$$

在基于字的二元模型中, 当前汉字只依赖前一个汉字, 因此整句概率可以近似写为

$$P(W) = P(w_1) \prod_{i=2}^n P(w_i | w_{i-1})$$

因此，拼音输入法的核心就变成：在每个拼音位置对应的候选汉字集合中，找到一条使上式最大的路径。

(2) 公式推导与概率估计

在训练阶段，程序从语料中统计单字频数和双字频数。设单字频数为 $C(w)$ ，二元频数为 $C(u,v)$ ，则可以估计：

$$P(w) = \frac{C(w)}{\sum_{x \in \mathcal{V}} C(x)}$$

$$P(v | u) = \frac{C(u,v)}{C(u)}$$

其中， $\mathcal{V}$ 表示合法汉字集合。

在实际计算中，如果直接连乘大量概率值，会出现数值下溢问题。因此，本实验将概率乘积转化为对数求和。由于对数函数是单调递增的，最大化原概率等价于最大化对数概率：

$$\hat{W} = \arg \max_W [\log P(w_1) + \sum_{i=2}^n \log P(w_i | w_{i-1})]$$

再等价于最小化负对数概率

$$\hat{W} = \arg \min_W [-\log P(w_1) + \sum_{i=2}^n (-\log P(w_i | w_{i-1}))]$$

这样，可以将路径的转移代价设定成对应的负对数概率，每条候选路径的总代价就是各位置转移代价之和。寻找最优句子就等价于寻找总代价最小的路径。

为了防止概率为0，采用平滑方法，将二元条件概率与一元概率进行线性插值。设平滑参数为 λ ，则平滑后的条件概率可以写为：

$$P(w_i | w_{i-1}) = (1 - \lambda)P(w_i | w_{i-1}) + \lambda P(w_i)$$

当二元统计可靠时，模型主要依赖二元概率；当某个二元组较少见或未出现时，一元概率可以提供一定补偿。

(3) 算法原理

采用 Viterbi 动态规划算法寻找总代价最小的路径。

设第*i*个位置选择候选汉字*w*时，前*i*个位置的最小总代价为 $\delta_i(w)$ ，则初始状态为

$$\delta_1(w) = -\log P(w)$$

状态转移方程为

$$\delta_i(w) = \min_u [\delta_{i-1}(u) + (-\log P'(w | u))]$$

u 遍历第*i*-1个位置的所有候选汉字。动态规划过程中，保存每个候选字的当前最小代价和使该代价达到最优的前驱节点。最终在最后一个位置选取总代价最小的状态，再通过回溯即可恢复整句最优汉字序列。

(4) 算法流程概述

本实验中基于字的二元模型拼音输入法整体流程如下：

1. 读取拼音汉字表和合法汉字表；
2. 遍历训练语料，统计单字频数和二元频数；
3. 对输入的每一行拼音序列按空格切分，得到拼音音节序列；
4. 根据拼音表为每个音节生成候选汉字集合；
5. 使用 Viterbi 算法进行动态规划，计算最优路径；

6. 回溯得到最终汉字句子并输出。

2.实验效果

(1) 生成词频表的时间和大小

- 生成单字频数表与二元频数表的总时间：168.5990s
- 单字频数表中不同汉字数量：6482
- 二元频数表中不同二元组数量：2824050

(2) 在测试样例上的准确率

基于sina_news_gbk训练，在给定测试样例上的结果如下：

指标	数值
字准确率	83.94%
句准确率	38.72%

(3) 生成所有测试样例的总时间

生成全部测试样例的总时间为：2.76s

3.例子分析

1. 效果较好的样例分析

对于训练语料中较常见的搭配、固定短语以及高频表达，二元模型通常具有较好的效果。这是因为这些汉字组合在语料中出现次数较多，其二元转移概率较高，模型更容易在动态规划时选出正确路径。

本实验中效果较好的样例如下：

- 人民群众喜闻乐见
- 青山绿水就是金山银山
- 中国共产党人的初心和使命是为中国人民谋幸福为中华民族谋复兴
- 经济建设和文化建设突出了十八大精神的重要性
- 机动车驾驶员培训手册
- 智能拼音输入法
- 我们都有光明的未来
- 无论我们遇到什么困难
- 机器学习是当下非常火热的技术
- 人工智能技术发展迅猛

这些句子效果较好的原因主要有以下几点：

1. 这类句子中包含大量高频搭配或固定短语，例如“机器学习”“人工智能”“金山银山”等，相邻字之间的二元转移概率较高，因此模型更容易选出正确路径。

2. 一些句子本身具有较强的语言模板特征，例如“我们都有.....的未来”“无论我们遇到什么困难”，其局部上下文已经足以支持二元模型做出正确判断。
3. 对于“机动车驾驶员培训手册”“智能拼音输入法”这类结构较规整、歧义较小的名词性短语，虽然整体未必是最高频句子，但每一对相邻汉字之间的搭配都比较稳定，因此模型也能较好恢复。
4. 语料以新闻报道为主，因此对于例如“人民群众”“中华民族”“经济建设”“十八大精神”“人工智能”等新闻报道和评论文章中较常出现的词语或搭配，模型判断的效果比较好

2. 效果较差的样例分析

对于生僻词、低频词、多音字、同音字密集句子以及需要更长距离上下文的信息，二元模型往往效果较差。这是因为二元模型只考虑前一个字，无法充分表达更长距离的语义依赖。

本实验中效果较差的样例如下：

- 正确句子：马少平老师如是说
模型输出：么少平老师如是说
- 正确句子：成本压力也不容小觑
模型输出：成本压力也不容小区
- 正确句子：北京大学比清华大学好
模型输出：北京大学毕清华大学好
- 正确句子：戴口罩防止交叉感染
模型输出：大口罩防止交叉感染

可能原因包括：

1. 由于二元词模型的局限性，对于一些语法功能词如“比”识别能力差，所以很难产生“.....比.....”这样的句式
2. 受限于二元模型，对于两字以上的词组识别效果差，比如“不容小觑”再模型中会倾向于分开识别为“不容”和“小区”两个高频二字词
3. 低频专有名词涉及较少，识别效果差。例如“马少平”属于特定人名
4. 语料未对多音字进行专门标注，仅依赖字频和上下文统计，模型对多音字处理能力有限。比如“大”字同时也有读音“dai”，导致“戴口罩”被误判为“大口罩”

四、思考题（必做）

1. 语料编码

字符集与适用范围：GBK 是本地化编码，主要面向中文；UTF-8 是国际通用编码（基于 Unicode），支持全球几乎所有已知语言的字符。

编码长度与存储：GBK 对中文字符固定使用2个字节，英文1个字节；UTF-8 是变长编码，常用中文字符占 3 个字节，英文占 1 个字节。

兼容性：GBK 在多语言混合或跨国系统环境下极易出现乱码；UTF-8 跨语言、跨平台兼容性更好，是现代互联网和软件开发的首选标准。

除 GBK 和 UTF-8 外，还存在其他编码方式。例如：

GB18030：国家标准字符集编码，兼容 GBK，覆盖字符更全面，适用于需要更完整中文字符支持的场景

2. 算法设计

设输入句子的长度为 n ，字库大小为 V ，并且汉字在读音上均匀分布，读音数量有限且为常数 C 。则每个拼音对应的候选汉字数约为 $\frac{V}{C}$ ， C 为常数，故其数量级可记为 $O(V)$ 。

在 Viterbi 算法中，对第 i 个位置的每个候选汉字，都需要枚举第 $i-1$ 个位置的所有候选汉字进行转移。每一层可认为由 $O(V)$ 个候选汉字因此每一层状态转移的时间复杂度为：

$$O(V \times V) = O(V^2)$$

总共有 n 个位置，所以总时间复杂度为：

$$O(nV^2)$$

空间复杂度方面，需要保存所有位置的动态规划得分和前驱信息，则每一层需要储存 $O(V)$ 个状态，共 n 层，因此空间复杂度为：

$$O(nV)$$

3. 输入输出

代码 A：

每接收到一个输入，就立即执行一次输出。因此在终端交互中，输入与输出会交替出现

当输入 1 后立即显示 1，输入 2 后再立即显示 2。输出过程与输入过程交织在一起。

代码 B：

程序先不断读取输入并保存到列表中，等所有输入结束后，再按顺序逐个输出；

因此在输入阶段不会立刻出现输出，最终会统一打印为标准答案形式。

两者都是正确的，oj会将标准输出写入到文件中，最后对比

五、不同语料的训练效果

实验中分别测试了不同语料配置下二元模型的性能，结果如下：

语料配置	字准确率	句准确率
同时使用三种语料	82.96%	35.73%
只用 sina_news	83.94%	38.72%
只用 SMP	76.91%	26.55%
同时使用 sina_news 和 SMP	84.05%	39.12%
只用 wiki_zh	78.57%	24.75%

结果分析

从实验结果可以看出，不同语料对模型效果的影响非常明显，而且并不是语料种类越多、规模越大，效果就一定越好。

综合来看，语料库训练效果上：sina_news > SMP > wiki_zh；但数据规模上：wiki_zh > sina_news > SMP

- 单独使用 sina_news 的效果明显优于单独使用 SMP 和 wiki_zh
- 单独使用 SMP 的效果明显下降
- 单独使用 wiki_zh 的效果也不理想,甚至将 wiki_zh 加入到训练语料中会造成准确率下降

原因分析：

- 测试集主要是一些新闻语句和日常用语，比较符合sina_news和SMP两个语料库的风格。而wiki_zh的百科风格和测试集的风格相差较大，所以效果差，加入训练甚至会造成准确率下降
- wiki_zh中的用语对于一些国内新闻常用的语言涉及较少，比如使用wiki_zh训练会导致“以西晋平统治委总书记得当中央”，“青山绿水就是金山阴山”这种错误
- 语料库的大小对训练的效果有影响，单独使用SMP的效果较差，但是结合使用sina_news和SMP的效果比单独使用更好

六、基于字的三元模型

1.基本思路

二元模型在解码时只考虑前一个汉字对当前汉字的影响，即近似使用：

$$P(w_i | w_{i-1})P(w_i | w_{i-1})P(w_i | w_{i-1})$$

而三元模型在预测当前位置汉字时，同时考虑前面两个汉字，使用：

$$P(w_i | w_{i-2}, w_{i-1})$$

可近似认为 $P(S) \approx \prod_i P(w_i | w_{i-2}, w_{i-1})$

在训练阶段，在原有单字频数表、二元频数表的基础上，额外统计了三元组频数表，即 (w_{i-2}, w_{i-1}, w_i) 的出现次数。

在概率计算时，采用了回退式线性插值。也就是说，三元概率由三部分共同决定：

$$P(w_i | w_{i-2}, w_{i-1}) = \lambda_3 P_{ML}(w_i | w_{i-2}, w_{i-1}) + \lambda_2 P(w_i | w_{i-1}) + \lambda_1 P(w_i)$$

$$\lambda_1 + \lambda_2 + \lambda_3 = 1。$$

其中：

- 第一项是三元条件概率；
- 第二项是在三元信息不足时回退到二元概率；
- 第三项再回退到一元概率。

这样的好处是：当某个三元组合在训练语料中没有出现过时，模型不会直接得到 0 概率，而是退回到较低阶模型

求解时仍采用 Viterbi 算法，只是状态转移代价从二元概率改为三元概率。

令 $g_i(w_{i-1}, w_i)$ 表示在位置 i 时, 第 $i - 1$ 个字为 w_{i-1} 且第 i 个字为 w_i 的所有可能路径中的最大概率。

动态规划转移方程:

$$g_i(w_{i-1}, w_i) = \max_{w_{i-2}} [g_{i-1}(w_{i-2}, w_{i-1}) \cdot P(w_i \mid w_{i-2}, w_{i-1})]$$

注意边界处理

- 当 $i = 1$ 时, 由于尚未形成二元或三元历史, 需直接按照**一元模型** (或仅依赖起始符的二元模型) 初始化概率。
- 当 $i = 2$ 时, 由于只有一个历史词, 需按照**二元模型**进行状态的初始化和转移。
- 当 $i \geq 3$ 时, 才正式启用上述的三元动态规划转移方程。

2.实验结果对比

我分别使用 `sina_news_gbk` 语料, 对二元模型和三元模型进行了训练和测试。参数采用代码中的默认参数, 未特别指定, 使用指令 `python main.py --model 3g < data/input.txt > data/output.txt` 运行

实验中, 二元模型与三元模型都基于同一套拼音汉字表和合法汉字表, 主程序通过参数选择加载 `2g` 或 `3g` 模型。

模型	训练语料	生成词频表时间 / s	词频表规模	测试时间 / s	字准确率	句准确率
二元字模型	sina_news	168.5990	单字 6482; 二元 2824050	2.7611	83.9351%	38.7226%
三元字模型	sina_news	387.0103	单字 6482; 二元 2824050; 三元 25200195	4.8966	90.8309%	61.0778%

从结果可以看出, 三元模型相比二元模型取得了明显提升:

- 字准确率从 83.94% 提升到 90.83%, 提高了 6.90 个百分点
- 句准确率从 38.72% 提升到 61.08%, 提高了 22.36 个百分点。

与此同时, 三元模型的代价也比较明显:

- 生成词频表时间约为二元模型的 2.30 倍
- 测试集预测时间约为二元模型的 1.77 倍
- 三元表条目数达到 25200195, 远大于二元表的 2824050, 说明其时间和空间开销显著增加。

3.具体例子分析

(1) 三元模型明显优于二元模型的例子

1

- 二元模型：**仅用**的小说很精彩
- 三元模型：“**金庸**的小说很精彩”

这里三元模型更好地利用了前后文，使专有名词识别有所改善。

2

- 二元输出：中国**陈文**应对贸易摩擦
- 三元输出：中国**沉稳**应对贸易摩擦

三元模型看到了“中国”作为主语，结合语境成功解码出正确的形容词“沉稳”（中国-沉稳-应对）

3

- 二元输出：认为为了谋取私利**时不惜一切大家的**
- 三元输出：认为为了谋取私利**是不惜一切代价的**
- **分析：**“不惜一切代价”是一个高频短语。二元模型由于视野局限，把 `dai jia de` 解码成了“大家的”。三元模型捕获到了“切-代-价”的转移，成功还原了这句话。

4

- 二元输出：清华慈真正成为**中国瓷器的主流**
- 三元输出：青花瓷真正成为**中国瓷器的主流**

二元模型极度依赖相邻两字的频率。由于语料库中“清华”的出现频率极高，二元模型强行拼凑出“清华慈”。三元模型则通过更长的上下文，成功识别出三字专有名词“青花瓷”

5

- 二元输出：阳光开朗大**南海**
- 三元输出：阳光开朗大**男孩**

“南海”是一个极高频的地名词，所以在二元模型中被误识别。三元模型通过“大男孩”词组和前面的“开朗”的约束成功识别

(2) 三元模型出错的例子

1

- 二元模型输出：我当场把这个电脑屏幕**吃掉**
- 三元模型输出：我当场把这个电脑屏幕**尺调**

二元模型识别到“吃掉”这个高频词成功输出正确句子

三元模型可能因为将前面的“屏幕”纳入了考量，由于新闻语言中屏幕和吃掉连用及其少见，所以反而将吃掉给排除掉了

- 二元输出：浙江温州江南皮革厂倒闭了
- 三元输出：浙江温州港南皮革厂倒闭了

二元模型识别到“江南”这一高频词从而成功识别，在三元模型中反而误判

七、调参

针对二元模型进行调参

调整lam

本实验固定 α 为0.05，调整 λ 进行实验

lam	字准确率	句准确率	预测时间/s
0.70	83.0755%	35.3293%	6.1785
0.80	83.7440%	37.7246%	5.8936
0.85	83.8968%	38.9222%	5.8964
0.90	83.9351%	38.7226%	6.1350
0.95	83.8777%	38.7226%	5.9820
0.99	84.0497%	39.5210%	6.9933
0.991	84.0115%	39.3214%	2.4765
0.995	83.9733%	39.3214%	2.5025
0.997	84.0306%	39.7206%	2.4953
0.999	84.0115%	39.7206%	2.4555

结果表明，随着 lam 增大，模型性能整体呈上升趋势，说明在当前拼音输入法任务中，前一个字提供的上下文信息明显比单字全局频率更重要。

在所有测试结果中，lam=0.997 时模型取得最佳综合表现：字准确率为 84.0306%，句准确率为 39.7206%。这说明提高二元概率权重能够有效增强模型对局部搭配关系的利用，从而改善预测效果。

进一步看，最优值出现在接近 1 的区域，而不是较小取值。这意味着对于当前任务，模型应当主要依赖二元上下文信息；但最优值又并非完全等于 1，说明仍需保留极少量一元概率作为兜底，以缓解二元统计不足或局部稀疏带来的不稳定问题。

调整 α

固定 lam=0.997，仅改变 alpha，分别测试 1e-6、0.001、0.01、0.05、0.1、0.5、1.0 七组取值

alpha	lam	字准确率	句准确率	预测时间/s
0.000001	0.997	84.0306%	39.7206%	2.4216
0.001	0.997	84.0306%	39.7206%	2.4501

alpha	lam	字准确率	句准确率	预测时间/s
0.01	0.997	84.0306%	39.7206%	2.4252
0.05	0.997	84.0306%	39.7206%	2.4475
0.10	0.997	84.0306%	39.7206%	2.4463
0.50	0.997	84.0306%	39.7206%	2.4455
1.00	0.997	84.0306%	39.7206%	2.4666

在上述范围内，模型的字准确率始终为 **84.0306%**，句准确率始终为 **39.7206%**，几乎没有任何变化。由此可见，在当前二元模型中，平滑参数 `alpha` 对最终预测结果影响很小

原因分析：

在 `sina_news` 语料上训练后，单字和二元统计已经较为充分，模型在解码时主要依赖前一个字提供的上下文信息，以及二元条件概率本身的差异

相比之下，平滑项更多只是对低频情况下的概率进行轻微修正，难以改变最终最优路径。

针对三元模型进行调参

第一次调整

编号	alpha	lam1	lam2	lam3	字准确率	句准确率	预测时间/s
1	0.01	0.05	0.10	0.85	90.8309%	61.0778%	16.3815
2	0.01	0.05	0.20	0.75	91.1366%	61.6766%	15.7141
3	0.01	0.10	0.20	0.70	90.9456%	60.8782%	15.7261
4	0.05	0.05	0.10	0.85	90.8309%	61.0778%	14.9710
5	0.05	0.05	0.20	0.75	91.1366%	61.6766%	14.8798
6	0.05	0.10	0.20	0.70	90.9456%	60.8782%	15.1495
7	0.10	0.05	0.10	0.85	90.8309%	61.0778%	15.3220
8	0.10	0.05	0.20	0.75	91.1366%	61.6766%	15.4890

分析：

我对三元字模型中的平滑参数 `alpha` 以及三元插值权重 `lam1`、`lam2`、`lam3` 进行了实验。

本次实验固定训练语料为 `sina_news_gbk`，在相同测试集上比较不同参数组合下的字准确率、句准确率和预测时间。实验结果表明，在测试的 `alpha=0.01、0.05、0.1` 范围内，模型性能几乎没有变化；而不同的 `lam1、lam2、lam3` 组合则会明显影响结果。

其中，最佳参数组合为：

$$\alpha = 0.05, \quad \lambda_1 = 0.05, \quad \lambda_2 = 0.20, \quad \lambda_3 = 0.75$$

在该参数下，模型字准确率达到 **91.1366%**，句准确率达到 **61.6766%**，均优于默认参数组合 (0.05,0.10,0.85) 下的结果。

从结果可以看出，适当降低三元概率权重、提高二元回退权重，可以提升模型效果。原因在于三元模型虽然能利用更长的上下文信息，但三元统计也更加稀疏，若权重过大，模型会对低频三元组过于敏感，导致泛化能力下降。提高二元权重后，模型在遇到不稳定三元统计时能够回退到更可靠的二元信息。

另一方面，增大一元权重会使模型更倾向于选择全局高频字，而减弱上下文约束，从而导致准确率下降。这说明在三元模型中，一元概率适合作为兜底项，但不应占据过大比重。

此外，本实验中 `alpha` 在小范围变化时几乎不影响最终准确率，说明在当前较大规模语料条件下，模型性能更多由上下文统计与插值策略决定，而不是由微小的平滑差异决定。

综合来看，本次调参实验说明：三元模型并非三元权重越高越好，而是需要在高阶上下文信息与低阶统计稳定性之间进行平衡。

进一步调整 α

本轮实验固定三元模型的较优插值权重：

`lam1 = 0.05`、`lam2 = 0.20`、`lam3 = 0.75`

只调整平滑参数 `alpha`，取值为：

`1e-6`、`0.001`、`0.01`、`0.05`、`0.1`、`0.5`、`1.0`

实验结果表明，在上述范围内，模型的字准确率始终为 91.1366%，句准确率始终为 61.6766%，几乎没有任何变化；预测时间仅有极小波动，大约在 5.17s 到 5.36s 之间

alpha	lam1	lam2	lam3	字准确率	句准确率	预测时间/s
0.000001	0.05	0.20	0.75	91.1366%	61.6766%	5.1729
0.001	0.05	0.20	0.75	91.1366%	61.6766%	5.1650
0.01	0.05	0.20	0.75	91.1366%	61.6766%	5.2626
0.05	0.05	0.20	0.75	91.1366%	61.6766%	5.2242
0.10	0.05	0.20	0.75	91.1366%	61.6766%	5.2510
0.50	0.05	0.20	0.75	91.1366%	61.6766%	5.3288
1.00	0.05	0.20	0.75	91.1366%	61.6766%	5.3613

这说明：

对当前三元字模型而言，`alpha` 在较大范围内都不是影响性能的主要因素。

分析原因：

首先，训练语料规模较大，常见汉字及其二元、三元组合已经具有较充分的统计信息，因此模型解码时主要依赖上下文概率本身，而不是依赖平滑项来修正大量低频事件。

其次，`alpha` 仅影响低频情况下的概率分配，而不会改变模型对不同阶数统计信息的总体依赖关系；相比之下，插值权重 `lam1`、`lam2`、`lam3` 直接决定模型更偏向三元、二元还是一元统计，因此对性能影响更显著。

八、其他参数测试

模型	字准确率	句准确率	CER (字符错误率)	AED (平均句编辑距离)	WES (错误句平均错误字数)	LMR (句长匹配率)
三元模型	91.1366%	61.6766%	9.1691%	0.9581	2.4615	100.0000%

引入了字符错误率（CER）、平均句编辑距离（AED）、错误句平均错误字数（WES）和句长匹配率（LMR）作为补充评价指标。

实验结果表明，该模型的 **CER 为 9.1691%**，说明从整体上看，预测结果与标准答案之间仅存在较小比例的字符级差异

AED 为 0.9581，表示平均每个句子只需修改不到 1 个字即可得到正确结果，说明模型输出整体上已经与标准答案非常接近

WES 为 2.4615，表示在预测错误的句子中，平均每句约错 2.46 个字，说明模型一旦出错，通常也是局部偏差，而不是整句完全错误

LMR 为 100%，说明所有输出句子的长度都与标准答案一致，模型不会产生明显的插入或删除错误。

实验感受

用时：我从周五的下午做到了周日的凌晨做完了本次作业，用时大概20多个小时

工作量：感觉工作量还是蛮大的，主要体现在：

1. 理解题目要求和相关算法的学习
2. 整理项目结构，学习读取数据，清理数据
3. 实现二字、三字模型，设计并实现相关的算法和数据结构
4. 进行测试，特别是为了调参额外写了测试脚本

感受：

1. 实验原理看起来很简单，但是实现起来会遇到许多奇怪的问题，需要考虑很多细节。具体的参数调整等也有很多需要考虑的问题。所以基本的代码能力，实验调优能力很重要
2. python能力得到了锻炼，学习到了很多新的用法
3. 实际的实现往往是不可兼得的，将模型提升为三元固然可以提升准确率，但是也引入了更多信息，需要更多时间和内存。

失败的尝试：

一开始我以为训练语料越多效果就会越好，但是引入了一个大的语料库之后准确率反而下降。所以语料不是越多越好，还要考虑语料和应用场景的匹配度

我曾经尝试引入动态lam和beam search，结果效果反而变差，说明理论和实际应用之间是由差距的建议

1. 思考题的第1题和第3题感觉和我们的课程关系不是很大，引入一些更相关的思考题是不是更好

2. 可不可以做一些引导文档用于讲解实验中的一些关键点，比如说viterbi算法，三元模型等，降低同学们的学习成本